

দিন ৩

Hash Table (Hash Map ও Set)

ইন্টারভিউয়ের সবচেয়ে শক্তিশালী অস্ত্র — $O(n^2)$ -কে $O(n)$ বানানোর জাদু

আজ যা শিখবো

- Hash Table কী, Hash Function কীভাবে কাজ করে
- Collision এবং তার সমাধান (Chaining, Open Addressing)
- কেন lookup গড়ে $O(1)$ আর কখন $O(n)$
- PHP-তে Hash Map ও Set ব্যবহার
- Frequency Count ও Sliding Window প্যাটার্ন
- ৫টি গুরুত্বপূর্ণ কোডিং প্রশ্ন

কোর্স ভিডিও: ২২-২৩

দিন শেষে তুমি পারবে

- Hash table ভেতরে কীভাবে কাজ করে ব্যাখ্যা করতে পারবে
- যেকোনো "গুনতে হবে / আগে দেখেছি কি না" প্রশ্নে hash map ব্যবহার করতে পারবে
- Anagram, Duplicate, First Unique, Longest Substring নিজে লিখতে পারবে

স্ট্যান্ডি শিডিউল (প্রায় ৫-৬ ঘণ্টা)

১ ঘণ্টা	Hash table কনসেপ্ট + collision
১ ঘণ্টা	Contains Duplicate + Valid Anagram
১ ঘণ্টা	First Unique + Group Anagrams
১.৫ ঘণ্টা	Longest Substring (Sliding Window)
৪৫ মিনিট	থিওরি + প্র্যাকটিস + চিট শিট

অধ্যায় ১: Hash Table কী?

Hash Table এমন একটা ডেটা স্ট্রাকচার যেখানে **key** দিয়ে **value** রাখা যায়, আর key দিয়ে খুঁজলে গড়ে **O(1)** সময়ে পাওয়া যায়। যেমন "rahim" → 01711..., "apple" → 120।

বাস্তব উদাহরণ: হোটেলের চাবির বোর্ড

রিসেপশনে ১০০টা চাবি এলোমেলো একটা বাক্সে থাকলে রুম ৪২-এর চাবি খুঁজতে সব ঘাঁটতে হবে (O(n))। কিন্তু বোর্ডে প্রতিটা হুকে রুম নম্বর লেখা থাকলে সরাসরি ৪২ নম্বর হুকে হাত দাও (O(1))। Hash function হলো সেই নিয়ম যেটা বলে "কোন key কোন হুকে যাবে।"

ভেতরে কীভাবে কাজ করে?

- ভেতরে একটা সাধারণ array থাকে — প্রতিটা ঘরকে বলে **bucket**
- Hash function** key থেকে একটা সংখ্যা বানায়, তারপর **% size** করে bucket index বের করে
- সেই bucket-এ value রাখে। খোঁজার সময় একই hash হিসাব করে সরাসরি সেই bucket-এ যায়

PHP

```
// খুব সহজ একটা hash function (শেখার জন্য)
function simpleHash(string $key, int $size): int {
    $sum = 0;
    for ($i = 0; $i < strlen($key); $i++) {
        $sum += ord($key[$i]); // প্রতিটা অক্ষরের ASCII মান যোগ
    }
    return $sum % $size; // bucket index
}

echo simpleHash("cat", 7); // (99+97+116) % 7 = 312 % 7 = 4
echo " " . simpleHash("dog", 7); // (100+111+103) % 7 = 314 % 7 = 6
echo " " . simpleHash("act", 7); // একই অক্ষর → আবার 4! (collision)
```

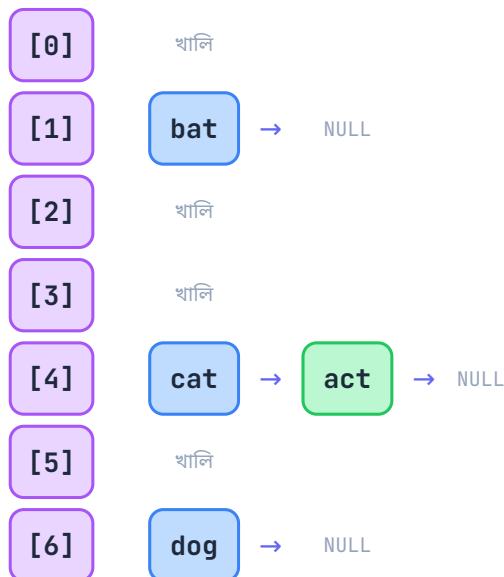
Collision — দুইটা key একই bucket-এ গেলে?

উপরের উদাহরণে "cat" আর "act" দুটোই bucket 4-এ যায়। এটাকে **collision** বলে। দুইটা প্রধান সমাধান:

১. Chaining (Separate Chaining)

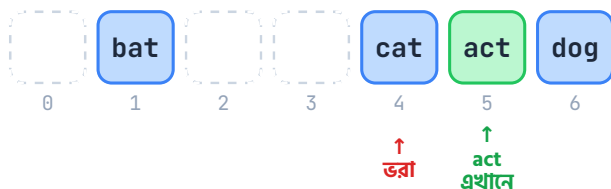
প্রতিটা bucket-এ একটা linked list রাখো। একই bucket-এ গেলে list-এ যোগ করো।

size = 7 এর hash table। "act" collision হয়ে "cat"-এর পেছনে chain-এ বসলো।



২. Open Addressing (Linear Probing)

Bucket ভরা থাকলে পরের খালি bucket খোঁজো (4 ভরা → 5 দেখো → খালি, সেখানে রাখো)।



Complexity — কেন "গড়ে" $O(1)$?

অপারেশন	গড় (Average)	সবচেয়ে খারাপ (Worst)
Insert	$O(1)$	$O(n)$
Search / Lookup	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

ভালো hash function key-গুলো সব bucket-এ সমানভাবে ছড়িয়ে দেয়, তাই প্রতিটা chain খুব ছোট থাকে → $O(1)$ । কিন্তু খারাপ hash function-এ সব key একই bucket-এ গেলে chain-টা n লম্বা হয়ে যায় → $O(n)$ ।

Load Factor ও Rehashing

Load factor = element সংখ্যা ÷ bucket সংখ্যা। এটা বেশি হলে (যেমন 0.75-এর বেশি) collision বাড়ে। তখন hash table bucket সংখ্যা দ্বিগুণ করে সব element নতুন করে বসায় (rehashing) — এটা $O(n)$, কিন্তু কদাচিৎ হয়, তাই গড়ে insert তবুও $O(1)$ (amortized)।

অধ্যায় ২: PHP-তে Hash Map ও Set

PHP-এর associative array-ই আমাদের hash map। আলাদা কিছু import করতে হয় না।

PHP

```
// ---- Hash Map ----
$ages = [];
$ages["rahim"] = 25;           // insert      0(1)
$ages["karim"] = 30;
echo $ages["rahim"];           // lookup      0(1) → 25
if (isset($ages["karim"])) echo " karim আছে"; // exists?  0(1)
unset($ages["rahim"]);         // delete      0(1)
echo " size=" . count($ages);  // 1

// ---- Frequency count (সবচেয়ে বেশি ব্যবহৃত প্যাটার্ন) ----
$freq = [];
foreach (str_split("banana") as $ch) {
    $freq[$ch] = ($freq[$ch] ?? 0) + 1; // না থাকলে 0 ধরো
}
echo "\n" . json_encode($freq);        // {"b":1,"a":3,"n":2}

// ---- Set (শুধু key ব্যবহার করো, value = true) ----
$seen = [];
$seen[5] = true;
echo "\n" . (isset($seen[5]) ? "5 দেখেছি" : "দেখিনি");
```

PHP-এর ফাঁদ

- `in_array($x, $arr)` হলো **O(n)** — value-তে খোঁজে। Hash-এর O(1) সুবিধা পেতে সবসময় key হিসেবে রাখো আর `isset()` ব্যবহার করো।
- `array_key_exists()` -ও O(1), কিন্তু `isset()` দ্রুত (তবে value null হলে `isset` false দেয়)।
- PHP string key "5" নিজে থেকে integer 5 হয়ে যায় — সাধারণত সমস্যা হয় না, তবে জেনে রাখো।

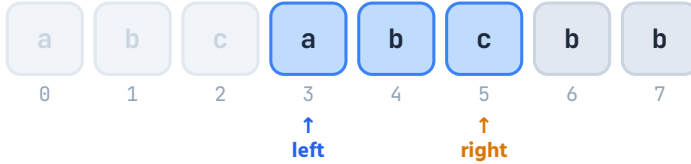
Hash Map চেনার সংকেত

প্রশ্ন যা দেখবে	কী ব্যবহার করবে
"আগে দেখেছি কি না" / duplicate	Set (<code>\$seen[\$x] = true</code>)
"কতবার এসেছে" / frequency	Count map (<code>\$freq[\$x]++</code>)
"জোড়া খোঁজো যার যোগফল..."	মান → index map (Two Sum)
"একই ধরনের জিনিস দলে ভাগ করো"	key → list map (Group Anagrams)
"সবচেয়ে লম্বা substring যাতে..."	Sliding window + map

অধ্যায় ৩: Sliding Window প্যাটার্ন

Array বা string-এর একটা পাশাপাশি অংশ (window) দুইটা pointer দিয়ে ধরো: **left** আর **right**। right দিয়ে window বড় করো, শর্ত ভাঙলে left দিয়ে ছোট করো। প্রতিটা element সর্বোচ্চ একবার ঢোকে আর একবার বের হয় → **O(n)**।

window = "abc" (index 3 থেকে 5)



কখন?

প্রশ্নে যদি থাকে "**subarray / substring**", "**পাশাপাশি (contiguous)**", "**সবচেয়ে লম্বা / সবচেয়ে ছোট**", "**k সাইজের**" — তাহলে sliding window!

প্রশ্ন 3.1

Contains Duplicate

Array-তে কোনো সংখ্যা একাধিকবার আছে কি না

Easy

LeetCode 217

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Hash set-এর সবচেয়ে সহজ ব্যবহার। তিনটা আলাদা সলিউশন ($O(n^2)$, $O(n \log n)$, $O(n)$) — trade-off আলোচনার জন্য দারুণ প্রশ্ন।

সমস্যাটা কী?

একটা integer array-তে কোনো মান অন্তত দুইবার থাকলে `true`, সব আলাদা হলে `false` রিটার্ন করো।

উদাহরণ

Input: [1, 2, 3, 1]

Output: true

Input: [1, 2, 3, 4]

Output: false

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

অনুষ্ঠানে গেটে অতিথিদের টিকিট নম্বর খাতায় লিখছো। নতুন কেউ এলে খাতায় দেখো — নম্বরটা আগেই লেখা থাকলে কেউ একই টিকিট দুইবার ব্যবহার করছে!

পদ্ধতি ১: Brute Force — সব জোড়া তুলনা

প্রতিটা i -এর জন্য পরের প্রতিটা j দেখো, সমান হলে `true`।

PHP

```
function containsDuplicateBrute(array $nums): bool {
    $n = count($nums);
    for ($i = 0; $i < $n; $i++) {
        for ($j = $i + 1; $j < $n; $j++) {
            if ($nums[$i] === $nums[$j]) return true;
        }
    }
    return false;
}
var_dump(containsDuplicateBrute([1, 2, 3, 1])); // true
```

Time: $O(n^2)$ Space: $O(1)$

পদ্ধতি ২: Sort করে পাশাপাশি দেখো

Sort করলে সমান মানগুলো পাশাপাশি চলে আসে। তারপর i আর $i+1$ তুলনা।

PHP

```
function containsDuplicateSort(array $nums): bool {
    sort($nums); // O(n log n)
    for ($i = 1; $i < count($nums); $i++) {
        if ($nums[$i] === $nums[$i - 1]) return true;
    }
    return false;
}
var_dump(containsDuplicateSort([1, 2, 3, 1])); // true
```

Time: $O(n \log n)$ Space: $O(1)$ (sort-এর space বাদে)

পদ্ধতি ৩: Hash Set — এক পাসে

1. খালি set নাও
2. প্রতিটা সংখ্যা: set-এ আগে থেকেই আছে → true
3. না থাকলে set-এ রাখো

PHP

```
function containsDuplicate(array $nums): bool {
    $seen = [];
    foreach ($nums as $x) {
        if (isset($seen[$x])) return true; // আগে দেখেছি!
        $seen[$x] = true;
    }
    return false;
}
var_dump(containsDuplicate([1, 2, 3, 1])); // true
var_dump(containsDuplicate([1, 2, 3, 4])); // false
```

Time: $O(n)$ Space: $O(n)$

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 $x=1$: set-এ নেই → রাখো



seen

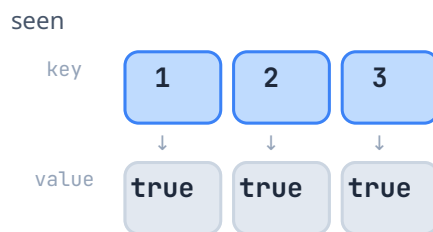
key

1

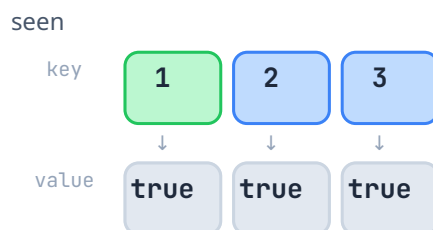
value

true

ধাপ 2 $x=2$: নেই $\rightarrow$ রাখো। $x=3$: নেই $\rightarrow$ রাখো



ধাপ 3 $x=1$: **set-এ আছে!** $\rightarrow$ true রিটার্ন



এজ কেস (এগুলো ভুলবে না)

- খালি array বা এক element $\rightarrow$ false

ইন্টারভিউতে যা বলবে

- তিনটা পদ্ধতিই বলো আর trade-off ব্যাখ্যা করো: "memory কম থাকলে sort পদ্ধতি, দ্রুততা দরকার হলে hash set।"
- শর্টকাট: `count($nums) !== count(array_unique($nums))` — কিন্তু ইন্টারভিউতে নিজে লিখে দেখাও।

প্রশ্ন 3.2

Valid Anagram

দুইটা শব্দ একই অক্ষর দিয়ে তৈরি কি না

Easy

LeetCode 242

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Frequency count প্যাটার্নের ক্লাসিক প্রশ্ন। String প্রশ্নে সবচেয়ে বেশি আসে।

সমস্যাটা কী?

দুইটা string `s` আর `t`। `t` যদি `s`-এর অক্ষরগুলো পুনর্বিন্যাস করে বানানো যায় (প্রতিটা অক্ষর ঠিক একই সংখ্যকবার), তাহলে `true`।

উদাহরণ

Input: `s = "anagram", t = "nagaram"`Output: `true`Input: `s = "rat", t = "car"`Output: `false`

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

"listen" আর "silent" — দুটো শব্দের অক্ষরের টুকরোগুলো একটা ব্যাগে রাখলে দুই ব্যাগ ছবছ একই হবে। আমরা প্রতিটা অক্ষর গুলে তুলনা করবো।

পদ্ধতি ১: দুটোকেই sort করে তুলনা

Anagram হলে sort করার পর দুই string ছবছ এক হবে।

PHP

```
function isAnagramSort(string $s, string $t): bool {
    if (strlen($s) !== strlen($t)) return false;
    $a = str_split($s); sort($a);
    $b = str_split($t); sort($b);
    return $a === $b;
}
var_dump(isAnagramSort("anagram", "nagaram")); // true
```

Time: $O(n \log n)$ Space: $O(n)$

পদ্ধতি ২: Frequency Count — এক map দিয়ে

1. দৈর্ঘ্য আলাদা হলে সরাসরি `false`
2. `s`-এর প্রতিটা অক্ষরে `count++`
3. `t`-এর প্রতিটা অক্ষরে `count--`

4. কোনো count 0 না হলে false

PHP

```
function isAnagram(string $s, string $t): bool {
    if (strlen($s) !== strlen($t)) return false;
    $count = [];
    for ($i = 0; $i < strlen($s); $i++) {
        $count[$s[$i]] = ($count[$s[$i]] ?? 0) + 1; // s-এর অক্ষর যোগ
        $count[$t[$i]] = ($count[$t[$i]] ?? 0) - 1; // t-এর অক্ষর বিয়োগ
    }
    foreach ($count as $c) {
        if ($c !== 0) return false;
    }
    return true;
}
var_dump(isAnagram("anagram", "nagaram")); // true
var_dump(isAnagram("rat", "car")); // false
```

Time: $O(n)$ Space: $O(1)$ — সর্বোচ্চ ২৬টা অক্ষরছোট হাতের ইংরেজি অক্ষর হলে map-এ সর্বোচ্চ ২৬টা key, তাই space $O(1)$ বলা যায়।

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 s = "anagram" গুনলে:

count (s)

key	a	n	g	r	m
value	3	1	1	1	1

ধাপ 2 t = "nagaram"-এর প্রতিটা অক্ষর বিয়োগ করলে:

count (s - t)

key	a	n	g	r	m
value	0	0	0	0	0

ধাপ 3 সব 0 → true। "rat" বনাম "car" হলে: r=0, a=0, t=+1, c=-1 → false

count

key	r	a	t	c
value	0	0	1	-1

এজ কেস (এগুলো ভুলবে না)

- দৈর্ঘ্য আলাদা → সাথে সাথে false
- খালি দুটো string → true
- বড়-ছোট হাতের অক্ষর আলাদা ধরবে কি না জিজ্ঞেস করো

ইন্টারভিউতে যা বলবে

- Follow-up আসতে পারে: "Unicode (যেমন বাংলা) হলে?" — array-র বদলে map ঠিক কাজ করবে, PHP-তে `mb_str_split()` ব্যবহার করো।

প্রশ্ন 3.3

First Unique Character in a String

প্রথম যে অক্ষর মাত্র একবার আছে

Easy

LeetCode 387

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

"দুই পাস" কৌশল শেখায়: প্রথম পাসে গোনো, দ্বিতীয় পাসে সিদ্ধান্ত নাও। অনেক প্রশ্নে লাগে।

সমস্যাটা কী?

একটা string-এ প্রথম যে অক্ষরটা পুনরাবৃত্তি হয়নি তার index রিটার্ন করো। না থাকলে -1।

উদাহরণ

Input: "leetcode"
Output: 0 // l একবারই আছে
Input: "loveleetcode"
Output: 2 // v
Input: "aabb"
Output: -1

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

ক্লাসের হাজিরা খাতায় কে কতবার নাম লিখেছে প্রথমে গুনে নাও। তারপর শুরু থেকে দেখো — প্রথম যার নাম মাত্র একবার, সেই উত্তর।

পদ্ধতি ১: প্রতিটা অক্ষরের জন্য পুরো string গোনো

প্রতিটা i-এর জন্য পুরো string ঘুরে দেখো তার মতো আর কেউ আছে কি না।

PHP

```
function firstUniqCharBrute(string $s): int {
    $n = strlen($s);
    for ($i = 0; $i < $n; $i++) {
        $unique = true;
        for ($j = 0; $j < $n; $j++) {
            if ($i !== $j && $s[$i] === $s[$j]) { $unique = false; break; }
        }
        if ($unique) return $i;
    }
    return -1;
}
echo firstUniqCharBrute("loveleetcode"); // 2
```

Time: $O(n^2)$ Space: $O(1)$

পদ্ধতি ২: দুই পাস — আগে গোনো, পরে খোঁজো

- পাস ১: প্রতিটা অক্ষর কতবার আছে map-এ গোনো
- পাস ২: শুরু থেকে দেখো — যার count 1, তার index রিটার্ন

PHP

```
function firstUniqChar(string $s): int {
    $freq = [];
    for ($i = 0; $i < strlen($s); $i++) {           // পাস ১
        $freq[$s[$i]] = ($freq[$s[$i]] ?? 0) + 1;
    }
    for ($i = 0; $i < strlen($s); $i++) {           // পাস ২
        if ($freq[$s[$i]] == 1) return $i;
    }
    return -1;
}
echo firstUniqChar("leetcode");           // 0
echo " " . firstUniqChar("loveleetcode"); // 2
echo " " . firstUniqChar("aabb");         // -1
```

Time: $O(n)$ Space: $O(1)$ — সর্বোচ্চ ২৬ অক্ষর

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ ১ পাস ১: "loveleetcode" গুনলাম

freq

key	l	o	v	e	t	c	d
value	2	2	1	4	1	1	1

ধাপ ২ পাস ২: index 0 "l" → 2, index 1 "o" → 2, index 2 "v" → **1!** উত্তর 2

l	o	v	e	l	e	e	t	c	o	d	e
0	1	2	3	4	5	6	7	8	9	10	11

↑
উত্তর

এজ কেস (এগুলো ভুলবে না)

- সব অক্ষর পুনরাবৃত্ত → -1
- এক অক্ষরের string → 0
- খালি string → -1

ইন্টারভিউতে যা বলবে

- কেন দুই পাস লাগে? এক পাসে জানা যায় না পরে অক্ষরটা আবার আসবে কি না।

প্রশ্ন 3.4

Group Anagrams

Anagram শব্দগুলোকে দলে ভাগ করো

Medium

LeetCode 49

কেন এটা গুরুত্বপূর্ণ?

"key বানিয়ে group করা" প্যাটার্ন — বাস্তব কাজেও (যেমন ডেটা grouping) খুব লাগে। Medium লেভেলের কমন প্রশ্ন।

সমস্যাটা কী?

String-এর একটা array দেওয়া আছে। যেগুলো একে অপরের anagram, তাদের একসাথে দলে রাখো।

উদাহরণ

Input: ["eat","tea","tan","ate","nat","bat"]

Output: [["eat","tea","ate"], ["tan","nat"], ["bat"]]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

প্রতিটা শব্দের অক্ষর sort করলে একটা "পরিচয়পত্র" পাওয়া যায়: eat, tea, ate সবার পরিচয় "aet"। একই পরিচয়পত্রের সবাইকে এক বাস্তবে রাখো।

পদ্ধতি ১: প্রতিটা জোড়া isAnagram দিয়ে চেক

প্রতিটা শব্দের জন্য আগের প্রতিটা দলের প্রথম শব্দের সাথে anagram কি না চেক করো। অনেক তুলনা লাগে।

PHP

```
function groupAnagramsBrute(array $strs): array {
    $groups = [];
    foreach ($strs as $w) {
        $placed = false;
        foreach ($groups as &$g) {
            if (count_chars($g[0], 1) == count_chars($w, 1)) { // একই অক্ষর-গণনা?
                $g[] = $w; $placed = true; break;
            }
        }
        unset($g);
        if (!$placed) $groups[] = [$w];
    }
    return $groups;
}

echo json_encode(groupAnagramsBrute(["eat","tea","tan","ate","nat","bat"]));
```

Time: $O(n^2 \cdot k)$ Space: $O(n \cdot k)$

n = শব্দ সংখ্যা, k = শব্দের দৈর্ঘ্য।

পদ্ধতি ২: Sorted key দিয়ে Hash Map

1. প্রতিটা শব্দের অক্ষর sort করে key বানাও
2. map[key]-এর list-এ শব্দটা যোগ করো
3. map-এর সব value রিটার্ন করো

PHP

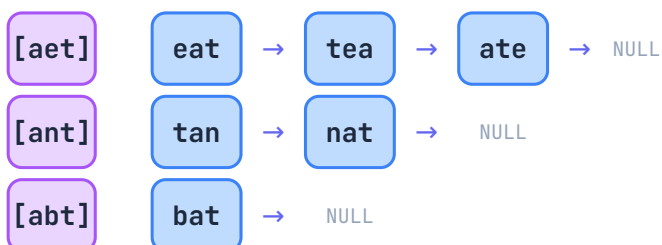
```
function groupAnagrams(array $strs): array {
    $map = [];
    foreach ($strs as $word) {
        $chars = str_split($word);
        sort($chars);
        $key = implode('', $chars); // "eat" → "aet"
        $map[$key][] = $word;      // একই key-র list-এ যোগ
    }
    return array_values($map);
}
echo json_encode(groupAnagrams(["eat", "tea", "tan", "ate", "nat", "bat"]));
// [["eat","tea","ate"],["tan","nat"],["bat"]]
```

Time: $O(n \cdot k \log k)$ Space: $O(n \cdot k)$

ড্রাই রান — ধাপে ধাপে দেখি

শব্দ	sorted key	map-এর অবস্থা
eat	aet	aet → [eat]
tea	aet	aet → [eat, tea]
tan	ant	aet → [eat, tea], ant → [tan]
ate	aet	aet → [eat, tea, ate], ant → [tan]
nat	ant	aet → [...], ant → [tan, nat]
bat	abt	aet → [3টা], ant → [2টা], abt → [bat]

শেষ অবস্থা: প্রতিটা key-র নিচে তার দল



এজ কেস (এগুলো ভুলবে না)

- খালি string "" → key-ও "", ঠিক কাজ করে
- একটাই শব্দ → [[শব্দ]]

ইন্টারভিউতে যা বলবে

- আরও ভালো key: sort-এর বদলে ২৬ সাইজের count array দিয়ে key বানাও ("1#0#0#..." → $O(n \cdot k)$)। এটা বললে বাড়তি নম্বর।

প্রশ্ন 3.5

Longest Substring Without Repeating Characters

পুনরাবৃত্তি ছাড়া সবচেয়ে লম্বা substring

Medium

LeetCode 3

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Sliding window + hash map-এর সবচেয়ে বিখ্যাত প্রশ্ন। প্রায় সব কোম্পানির ইন্টারভিউ লিস্টে থাকে।

সমস্যাটা কী?

একটা string `s`-এ এমন সবচেয়ে লম্বা substring-এর দৈর্ঘ্য বের করো যেখানে কোনো অক্ষর দুইবার নেই।

উদাহরণ

```
Input: "abcabcbb"
Output: 3 // "abc"
Input: "bbbbbb"
Output: 1 // "b"
Input: "pwwkew"
Output: 3 // "wke" ("pwke" substring না, subsequence)
```

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

একটা ট্রেনের বগিতে যাত্রী উঠছে (right)। নিয়ম: একই নামের দুইজন একসাথে থাকতে পারবে না। নতুন কেউ উঠলো যার নামের কেউ আগে থেকেই আছে — তখন সামনের দরজা (left) দিয়ে সেই পুরনো জন পর্যন্ত সবাইকে নামিয়ে দাও। প্রতিবার দেখো বগিতে সর্বোচ্চ কতজন ছিল।

পদ্ধতি ১: প্রতিটা শুরু থেকে যতদূর যাওয়া যায়

প্রতিটা `i` থেকে শুরু করে একটা set নিয়ে ডানে যাও; পুনরাবৃত্তি পেলে থামো। সর্বোচ্চ দৈর্ঘ্য রাখো।

PHP

```
function lengthOfLongestSubstringBrute(string $s): int {
    $n = strlen($s);
    $best = 0;
    for ($i = 0; $i < $n; $i++) {
        $seen = [];
        for ($j = $i; $j < $n; $j++) {
            if (isset($seen[$s[$j]])) break;
            $seen[$s[$j]] = true;
            $best = max($best, $j - $i + 1);
        }
    }
    return $best;
}

echo lengthOfLongestSubstringBrute("abcabcbb"); // 3
```

Time: $O(n^2)$ Space: $O(\min(n, \text{অক্ষর সংখ্যা}))$ **পদ্ধতি ২: Sliding Window + শেষ দেখা index-এর map**

1. map: অক্ষর $\rightarrow$ শেষবার কোন index-এ দেখেছি
2. right দিয়ে একে একে এগোও
3. অক্ষরটা window-এর ভেতরে ($\text{index} \geq \text{left}$) আগে থাকলে: $\text{left} = \text{সেই index} + 1$
4. map আপডেট, $\text{best} = \max(\text{best}, \text{right} - \text{left} + 1)$

PHP

```
function lengthOfLongestSubstring(string $s): int {
    $lastSeen = []; // অক্ষর => শেষ index
    $left = 0;
    $best = 0;
    for ($right = 0; $right < strlen($s); $right++) {
        $ch = $s[$right];
        if (isset($lastSeen[$ch]) && $lastSeen[$ch] >= $left) {
            $left = $lastSeen[$ch] + 1; // পুনরাবৃত্ত অক্ষরের পরে সরে যাও
        }
        $lastSeen[$ch] = $right;
        $best = max($best, $right - $left + 1);
    }
    return $best;
}

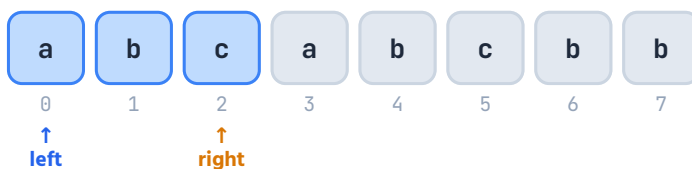
echo lengthOfLongestSubstring("abcabcbb"); // 3
echo " " . lengthOfLongestSubstring("bbbbbb"); // 1
echo " " . lengthOfLongestSubstring("pwwkew"); // 3
```

Time: $O(n)$ Space: $O(\min(n, \text{অক্ষর সংখ্যা}))$

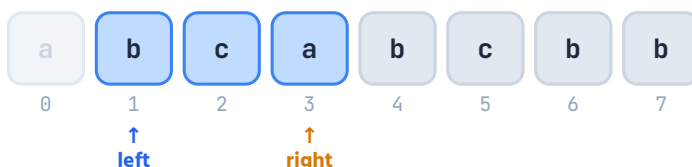
$\$lastSeen[\$ch] \geq \$left$ শর্তটা জরুরি — window-এর বাইরের পুরনো index দেখে left-কে পেছনে সরানো যাবে না।

ড্রাই রান — ধাপে ধাপে দেখি

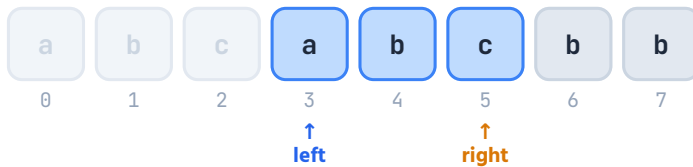
ধাপ 1 right=0..2: a, b, c — কোনো পুনরাবৃত্তি নেই। window "abc", best=3



ধাপ 2 right=3 "a": শেষ দেখা $0 \geq \text{left}(0) \rightarrow \text{left}=1$ । window "bca", best=3



ধাপ 3 right=4 "b": শেষ দেখা 1 → left=2। right=5 "c": শেষ দেখা 2 → left=3। window "abc"



ধাপ 4 right=6 "b": শেষ দেখা 4 → left=5। window "cb", দৈর্ঘ্য 2



ধাপ 5 right=7 "b": শেষ দেখা 6 → left=7। window "b"। শেষ! উত্তর best = 3



এজ কেস (এগুলো ভুলবে না)

- খালি string → 0
- সব একই অক্ষর → 1
- সব আলাদা → পুরো দৈর্ঘ্য
- space বা বিশেষ চিহ্নও অক্ষর হিসেবে গণ্য

ইন্টারভিউতে যা বলবে

- Substring (পাশাপাশি) আর Subsequence (ক্রম ঠিক, কিন্তু পাশাপাশি না-ও হতে পারে)-এর পার্থক্য পরিষ্কার করে বলো।
- বলো কেন $O(n)$: "left আর right দুটোই শুধু সামনে যায়, প্রত্যেকে সর্বোচ্চ n বার।"

থিওরি প্রশ্নোত্তর (ইন্টারভিউ স্টাইল)

প্রতিটা উত্তর ইন্টারভিউতে ৩০-৬০ সেকেন্ডে বলার মতো করে লেখা। মুখস্থ না করে নিজের ভাষায় বলার প্র্যাকটিস করো।

প্রশ্ন 1: Hash table কী এবং কীভাবে কাজ করে?

Key-value জোড়া রাখার ডেটা স্ট্রাকচার। ভেতরে একটা array থাকে; hash function key থেকে একটা index বানায়, সেই bucket-এ value রাখা হয়। খোঁজার সময় একই hash হিসাব করে সরাসরি সেই bucket-এ যায় — তাই গড়ে $O(1)$ ।

প্রশ্ন 2: Collision কী? কীভাবে handle করা হয়?

দুইটা আলাদা key একই bucket index পেলে collision। দুই সমাধান: (১) **Chaining** — প্রতিটা bucket-এ linked list, একই bucket-এর সবাই list-এ থাকে; (২) **Open addressing** — bucket ভরা থাকলে পরের খালি bucket খোঁজা (linear probing ইত্যাদি)।

প্রশ্ন 3: Hash table-এর worst case কেন $O(n)$?

যদি hash function খারাপ হয় বা সব key একই bucket-এ যায়, তাহলে সেই bucket-এর chain-এ n টা element জমে। তখন খুঁজতে পুরো chain ঘুরতে হয় — $O(n)$ ।

প্রশ্ন 4: Load factor আর rehashing কী?

Load factor = element সংখ্যা ÷ bucket সংখ্যা। এটা একটা সীমা (যেমন 0.75) পার হলে collision বাড়ে, তাই table-এর সাইজ বাড়িয়ে (সাধারণত দ্বিগুণ) সব element নতুন hash অনুযায়ী আবার বসানো হয় — rehashing। এটা $O(n)$, কিন্তু কদাচিৎ হয় বলে insert গড়ে $O(1)$ ।

প্রশ্ন 5: ভালো hash function-এর বৈশিষ্ট্য কী?

দ্রুত হিসাব হয়; একই key সবসময় একই মান দেয় (deterministic); key-গুলো সব bucket-এ সমানভাবে ছড়িয়ে দেয় যাতে collision কম হয়।

প্রশ্ন 6: Hash Map আর Hash Set-এর পার্থক্য?

Map-এ key → value জোড়া থাকে (যেমন নাম → বয়স)। Set-এ শুধু ইউনিক key থাকে, value নেই — শুধু "আছে কি না" জানতে। PHP-তে set বানাতে `$set[$x] = true` আর চেক করতে `isset($set[$x])`।

প্রশ্ন 7: Array বনাম Hash Map — কখন কোনটা?

Array: index (0,1,2...) দিয়ে ক্রমানুসারে ডেটা, sorted রাখা, মেমোরি কম। Hash map: যেকোনো key (string, id) দিয়ে দ্রুত খোঁজা, গণনা, duplicate চেক। Hash map-এ ক্রম গ্যারান্টি সাধারণত থাকে না (PHP-তে insertion order থাকে)।

প্রশ্ন 8: Sliding window কী?

Array/string-এর একটা পাশাপাশি অংশকে left আর right pointer দিয়ে ধরা। right দিয়ে বাড়াই, শর্ত ভাঙলে left দিয়ে কমাই। প্রতিটা element একবার ঢোকে, একবার বের হয় — $O(n)$ । Nested লুপের $O(n^2)$ এড়াতে ব্যবহার করি।

প্র্যাকটিস প্রবলেম (নিজে সলভ করো)

আগে নিজে ২০ মিনিট চেষ্টা করো। আটকে গেলে হিন্ট দেখো। তারপরও না পারলে সলিউশন দেখে, পরদিন আবার নিজে লেখো।

1. Top K Frequent Elements (Medium · LeetCode 347)

সবচেয়ে বেশিবার আসা k টা সংখ্যা। $[1,1,1,2,2,3]$, $k=2 \rightarrow [1,2]$

হিন্ট: Frequency map বানাও, তারপর `arsort()` দিয়ে count অনুযায়ী সাজিয়ে প্রথম k টা key নাও।

2. Intersection of Two Arrays (Easy · LeetCode 349)

দুইটা array-র কমন ইউনিক element।

হিন্ট: প্রথমটা set-এ রাখো; দ্বিতীয়টার যেগুলো set-এ আছে result-এ নাও (ডুপ্লিকেট এড়াতে result-ও set)।

3. Ransom Note (Easy · LeetCode 383)

magazine-এর অক্ষর দিয়ে ransomNote বানানো যায় কি না।

হিন্ট: magazine-এর frequency গোনো, note-এর প্রতিটা অক্ষরে count কমাও; নেগেটিভ হলে false।

4. Longest Consecutive Sequence (Medium · LeetCode 128)

$[100,4,200,1,3,2] \rightarrow 4 (1,2,3,4)$ — $O(n)$ -এ।

হিন্ট: সব set-এ রাখো। যে x-এর x-1 set-এ নেই, সেখান থেকেই শুধু গোনা শুরু করো।

5. Subarray Sum Equals K (Medium · LeetCode 560)

কয়টা পাশাপাশি subarray-র যোগফল k?

হিন্ট: Prefix sum + map: `map[sum - k]` কতবার দেখেছো সেটা count-এ যোগ করো।

6. Maximum Sum Subarray of Size K (Easy)

$[2,1,5,1,3,2]$, $k=3 \rightarrow 9 (5+1+3)$

হিন্ট: Fixed sliding window: প্রথম k টার যোগ, তারপর প্রতিবার নতুনটা যোগ আর পুরনোটা বিয়োগ।

চিট শিট — দিন ৩

PHP Hash Map কমান্ড

কাজ	কোড	Time
Insert/Update	<code>\$m[\$k] = \$v;</code>	$O(1)$
আছে কি না	<code>isset(\$m[\$k])</code>	$O(1)$
গণনা	<code>\$m[\$k] = (\$m[\$k] ?? 0) + 1;</code>	$O(1)$
Delete	<code>unset(\$m[\$k]);</code>	$O(1)$
সব key / value	<code>array_keys(\$m) / array_values(\$m)</code>	$O(n)$
Value অনুযায়ী sort (বড় আগে)	<code>arsort(\$m);</code>	$O(n \log n)$
এড়িয়ে চলো	<code>in_array(\$x, \$arr)</code>	$O(n)$

আজকের প্যাটার্ন

প্রশ্ন	কৌশল	Time / Space
Contains Duplicate	Set: আগে দেখেছি?	$O(n) / O(n)$
Valid Anagram	Count map (+ / -)	$O(n) / O(1)$
First Unique	দুই পাস: গোনো → খোঁজো	$O(n) / O(1)$
Group Anagrams	sorted key → list	$O(n \cdot k \log k)$
Longest Substring	Sliding window + lastSeen	$O(n) / O(k)$

Sliding Window টেমপ্লেট

PHP — টেমপ্লেট

```
$left = 0; $best = 0;
for ($right = 0; $right < $n; $right++) {
    // ১. $s[$right] window-এ যোগ করো
    while (/* window শর্ত ভাঙছে */) {
        // ২. $s[$left] window থেকে বাদ দাও
        $left++;
    }
    $best = max($best, $right - $left + 1); // ৩. উত্তর আপডেট
}
```

আজকের নিজেকে যাচাই

- ☐ Hash table ভেতরে কীভাবে কাজ করে বোর্ডে ঐকে বোঝাতে পারি
- ☐ Collision-এর দুই সমাধান বলতে পারি
- ☐ Valid Anagram আর Contains Duplicate না দেখে লিখতে পারি
- ☐ Longest Substring-এ left কেন সরে ব্যাখ্যা করতে পারি